

PostgreSQL Authentication Methods

Table of Contents

1. [Learning Objectives](#)
 2. [Authentication Architecture](#)
 3. [pg_hba.conf Deep Dive](#)
 4. [Trust Authentication](#)
 5. [MD5 Authentication](#)
 6. [SCRAM-SHA-256 Authentication](#)
 7. [Certificate Authentication](#)
 8. [LDAP Authentication](#)
 9. [RADIUS Authentication](#)
 10. [PAM Authentication](#)
 11. [Authentication Flow Diagram](#)
 12. [Common Mistakes](#)
 13. [Best Practices](#)
 14. [Performance Considerations](#)
 15. [Interview Questions](#)
 16. [Exercises and Solutions](#)
-

Learning Objectives

By the end of this module, you will be able to:

- Configure pg_hba.conf for different authentication scenarios
 - Choose the appropriate auth method for different client types
 - Set up SCRAM-SHA-256 as the production standard
 - Configure certificate-based authentication for highest security
 - Integrate PostgreSQL with LDAP for enterprise authentication
 - Trace the full authentication flow for debugging
-

Authentication Architecture

CLIENT CONNECTION REQUEST



pg_hba.conf Processing

1. Read records top to bottom
2. Find FIRST matching record:
 - connection type (local/host/hostssl)
 - database name
 - username
 - client IP address
3. Use that record's auth method
4. If auth succeeds → connect
5. If auth fails → reject (no fallback to next record)

6. If no record matches → reject with error



Authentication Method Handler

- |— trust: allow without password
- |— md5: old MD5 hash challenge
- |— scram-sha-256: modern SCRAM auth
- |— cert: TLS client certificate
- |— ldap: LDAP directory
- |— radius: RADIUS server
- |— pam: PAM service
- |— gss: Kerberos/GSSAPI
- |— sspi: Windows SSPI
- |— peer: OS user matching (local only)
- |— reject: always deny

pg_hba.conf Deep Dive

File Location and Format

```
# Find pg_hba.conf location
psql -c "SHOW hba_file;"
# /etc/postgresql/16/main/pg_hba.conf

# Or from data directory
ls $PGDATA/pg_hba.conf
```

Record Format

```
# TYPE DATABASE USER ADDRESS METHOD [OPTIONS]

# TYPE:
# local - Unix domain socket (no host)
# host - TCP/IP (SSL or non-SSL)
# hostssl - TCP/IP, SSL required
# hostnossl - TCP/IP, SSL not allowed
# hostgssenc - TCP/IP, GSSAPI encryption
# hostnogssenc - TCP/IP, no GSSAPI encryption

# DATABASE:
# all - all databases
# sameuser - database with same name as user
# samerole - databases owned by user's role
# @file.txt - read database list from file
# db1,db2 - specific databases
# replication - replication connections only

# USER:
```

```

# all          - all users
# +rolename    - role and its members
# @file.txt    - read user list from file
# user1,user2  - specific users

# ADDRESS (host/hostssl only):
# 192.168.1.0/24 - CIDR notation
# 192.168.1.0 255.255.255.0 - old netmask notation
# ::1/128      - IPv6 loopback
# all          - any address
# samehost     - server's IP addresses
# samenet      - server's subnets

```

Complete pg_hba.conf Example

```

# /etc/postgresql/16/main/pg_hba.conf
# PostgreSQL Client Authentication Configuration

#-----
# TYPE      DATABASE      USER      ADDRESS      METHOD

# Local connections (Unix socket – no network)
local      all            postgres   peer
local      all            all        scram-sha-256

# IPv4 loopback
host       all            all        127.0.0.1/32  scram-sha-256

# IPv6 loopback
host       all            all        ::1/128       scram-sha-256

# Replication connections (must use replication pseudo-database)
hostssl    replication    replicator 192.168.1.0/24  scram-sha-256

# Application server subnet (SSL required)
hostssl    all            appuser    10.0.1.0/24    scram-sha-256

# Analytics team (certificate auth for extra security)
hostssl    analytics_db   +analytics_role 10.0.2.0/24    cert

# Internal monitoring (password auth on internal network)
host       all            monitoring 192.168.10.5/32  scram-sha-256

# Admin access (require SSL + password)
hostssl    all            dbadmin    0.0.0.0/0      scram-sha-256

# LDAP for corporate users
host       all            @/etc/pg_users.txt 10.0.0.0/8      ldap
ldapserver=ldap.company.com ldapbasedn="dc=company,dc=com"

```

```
# Reject everything else
host      all             all             0.0.0.0/0          reject
```

pg_hba.conf Management

```
-- View loaded pg_hba.conf records
SELECT type, database, user_name, address, netmask, auth_method, options
FROM pg_hba_file_rules;

-- Reload after changes (no restart needed)
SELECT pg_reload_conf();
-- or: pg_ctl reload -D /var/lib/postgresql/16/main

-- Check effective authentication for a connection attempt
-- Look in logs after a failed connection
-- HINT: enable log_connections = on
```

Trust Authentication

Never use in production. Only for local development or isolated test environments.

```
# Trust: connect without any password
local  all  postgres  trust
host   all  all       127.0.0.1/32  trust

# Any OS user can connect as any PostgreSQL user with no password
psql -U postgres mydb # Succeeds with no password prompt
```

```
# DANGER: If trust is set for external IPs, anyone can log in as any user
# Even if no entry exists for specific hosts, be careful with:
# host all all 0.0.0.0/0 trust - NEVER DO THIS
```

MD5 Authentication

MD5 is the older password authentication method. The password is challenged as `md5(md5(password + username) + challenge_salt)`.

```
# pg_hba.conf
host  all  all  0.0.0.0/0  md5
```

```
-- Set MD5 password for user
CREATE ROLE apiuser WITH LOGIN PASSWORD 'MyPassword123';
-- Password stored as MD5 hash in pg_authid

-- View password hash type
SELECT rolname, passwd FROM pg_authid WHERE rolname = 'apiuser';
-- MD5 hash starts with 'md5'
```

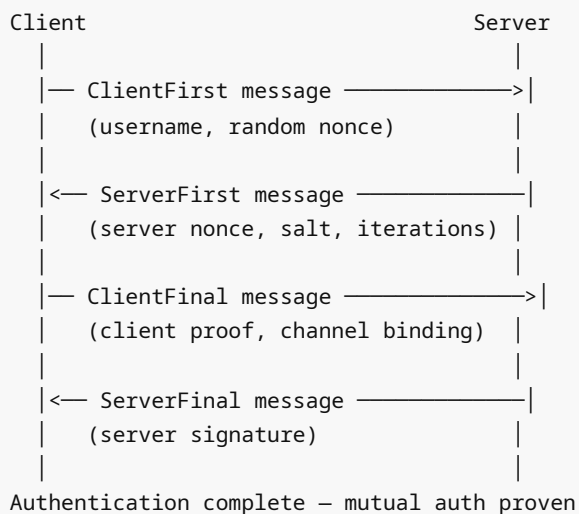
MD5 weaknesses:

- Vulnerable to offline dictionary attacks (weak hash)
- No protection against replay attacks at hash level
- Deprecated — use SCRAM-SHA-256 instead

SCRAM-SHA-256 Authentication

SCRAM-SHA-256 (Salted Challenge Response Authentication Mechanism) is the modern, recommended authentication method for PostgreSQL 10+.

How SCRAM Works



Configuration

```
# postgresql.conf
password_encryption = scram-sha-256    # Default in PG14+
```

```
-- Create user with SCRAM password
CREATE ROLE appuser WITH LOGIN PASSWORD 'SecurePassword123!';

-- Verify hash type
SELECT rolname, left(passwd, 12) AS hash_type
FROM pg_authid
WHERE rolname = 'appuser';
-- hash_type: SCRAM-SHA-256

-- Change existing user to SCRAM (if they were MD5)
ALTER ROLE olduser WITH PASSWORD 'NewPassword123!';
-- Re-encrypts using current password_encryption setting
```

```
# pg_hba.conf
hostssl all all 0.0.0.0/0 scram-sha-256
```

Migrating from MD5 to SCRAM

```
-- Step 1: Change server default
ALTER SYSTEM SET password_encryption = 'scram-sha-256';
SELECT pg_reload_conf();

-- Step 2: Force users to change passwords (re-encrypts as SCRAM)
-- Users must change own passwords, or admin changes them
ALTER ROLE user1 WITH PASSWORD 'SameOrNewPassword';

-- Step 3: Update pg_hba.conf
-- Change 'md5' to 'scram-sha-256' in entries

-- Step 4: Reload
SELECT pg_reload_conf();

-- Verify all users have SCRAM hashes
SELECT rolname, passwd FROM pg_authid
WHERE rolcanlogin = true
  AND passwd NOT LIKE 'SCRAM-SHA-256%'
  AND passwd IS NOT NULL;
-- Should return 0 rows when migration complete
```

Certificate Authentication

Certificate-based authentication uses TLS client certificates — no password needed. The client must present a valid certificate signed by a trusted CA.

CERTIFICATE AUTH FLOW:

Client	Server
— TLS ClientHello —————>	
<— TLS ServerHello + Cert ———	
— Client Certificate —————>	
(CN=username or mapped)	← Client proves identity by presenting cert
Server verifies cert against CA	
CN or subjectAltName maps to PG user	
Authentication complete (no password!)	

Setting Up Certificate Auth

```
# Step 1: Create CA
openssl genrsa -out ca.key 4096
openssl req -new -x509 -days 3650 -key ca.key -out ca.crt \
  -subj "/CN=PostgreSQL CA"

# Step 2: Create server certificate
```

```

openssl genrsa -out server.key 2048
openssl req -new -key server.key -out server.csr \
    -subj "/CN=db.company.com"
openssl x509 -req -days 365 -in server.csr -CA ca.crt -CAkey ca.key \
    -CAcreateserial -out server.crt

# Copy to PostgreSQL data directory
cp server.key server.crt ca.crt /etc/postgresql/16/main/
chmod 600 /etc/postgresql/16/main/server.key
chown postgres:postgres /etc/postgresql/16/main/server.*

# Step 3: Create client certificate (CN must match PG username)
openssl genrsa -out client.key 2048
openssl req -new -key client.key -out client.csr \
    -subj "/CN=appuser"          # CN = PostgreSQL username
openssl x509 -req -days 365 -in client.csr -CA ca.crt -CAkey ca.key \
    -CAcreateserial -out client.crt

```

```

# postgresql.conf
ssl = on
ssl_cert_file = 'server.crt'
ssl_key_file = 'server.key'
ssl_ca_file = 'ca.crt'          # CA that signed client certs

```

```

# pg_hba.conf
hostssl all appuser 0.0.0.0/0 cert
# 'cert' means: use TLS client certificate; CN must match username

```

```

# Connect with client certificate
psql "host=db.company.com dbname=mydb user=appuser \
    sslcert=client.crt sslkey=client.key \
    sslrootcert=ca.crt sslmode=verify-full"

```

Certificate Name Mapping

```

# /etc/postgresql/16/main/pg_ident.conf
# Map certificate CN to PostgreSQL username
# MAPNAME    SYSTEM-USERNAME    PG-USERNAME
certmap      john.doe@corp.com    appuser
certmap      jane.doe@corp.com    appuser2

```

```

# pg_hba.conf with mapping
hostssl all all 0.0.0.0/0 cert map=certmap

```

LDAP Authentication

LDAP authentication delegates password verification to an LDAP directory (Active Directory, OpenLDAP). The user still must exist as a PostgreSQL role.

LDAP AUTH FLOW:

```
Client --password--> PostgreSQL --bind--> LDAP Server
                                     (verifies password)
                                     <-- success/fail --
                                     <-- connect/reject --
```

Simple LDAP Configuration

```
# pg_hba.conf - Simple Bind
host all all 10.0.0.0/8 ldap
  ldapserver=ldap.company.com
  ldapbasedn="ou=People,dc=company,dc=com"
  ldapprefix="uid="
  ldapsuffix=",ou=People,dc=company,dc=com"
```

LDAP Search+Bind (more flexible)

```
# Search first, then bind with found DN
host all all 10.0.0.0/8 ldap
  ldapserver=ldap.company.com
  ldapbasedn="ou=People,dc=company,dc=com"
  ldapsearchattribute=uid
  ldapbinddn="cn=pgbind,dc=company,dc=com"
  ldapbindpasswd="bindpassword"
```

LDAP with TLS (LDAPS)

```
host all all 10.0.0.0/8 ldap
  ldapserver=ldap.company.com
  ldapport=636
  ldaptls=1
  ldapbasedn="ou=People,dc=company,dc=com"
  ldapprefix="uid="
  ldapsuffix=",ou=People,dc=company,dc=com"
```

```
-- PostgreSQL role must exist (LDAP only verifies password)
-- Create roles without passwords (password comes from LDAP)
CREATE ROLE "john.doe" WITH LOGIN; -- No password needed
```

RADIUS Authentication

RADIUS authentication delegates to a RADIUS server for password verification.

```
# pg_hba.conf
host all all 10.0.0.0/8 radius
```

```
radiusservers="radius1.company.com,radius2.company.com"
radiussecrets="shared_secret"
radiusports="1812"
radiusidentifiers="PostgreSQL"
```

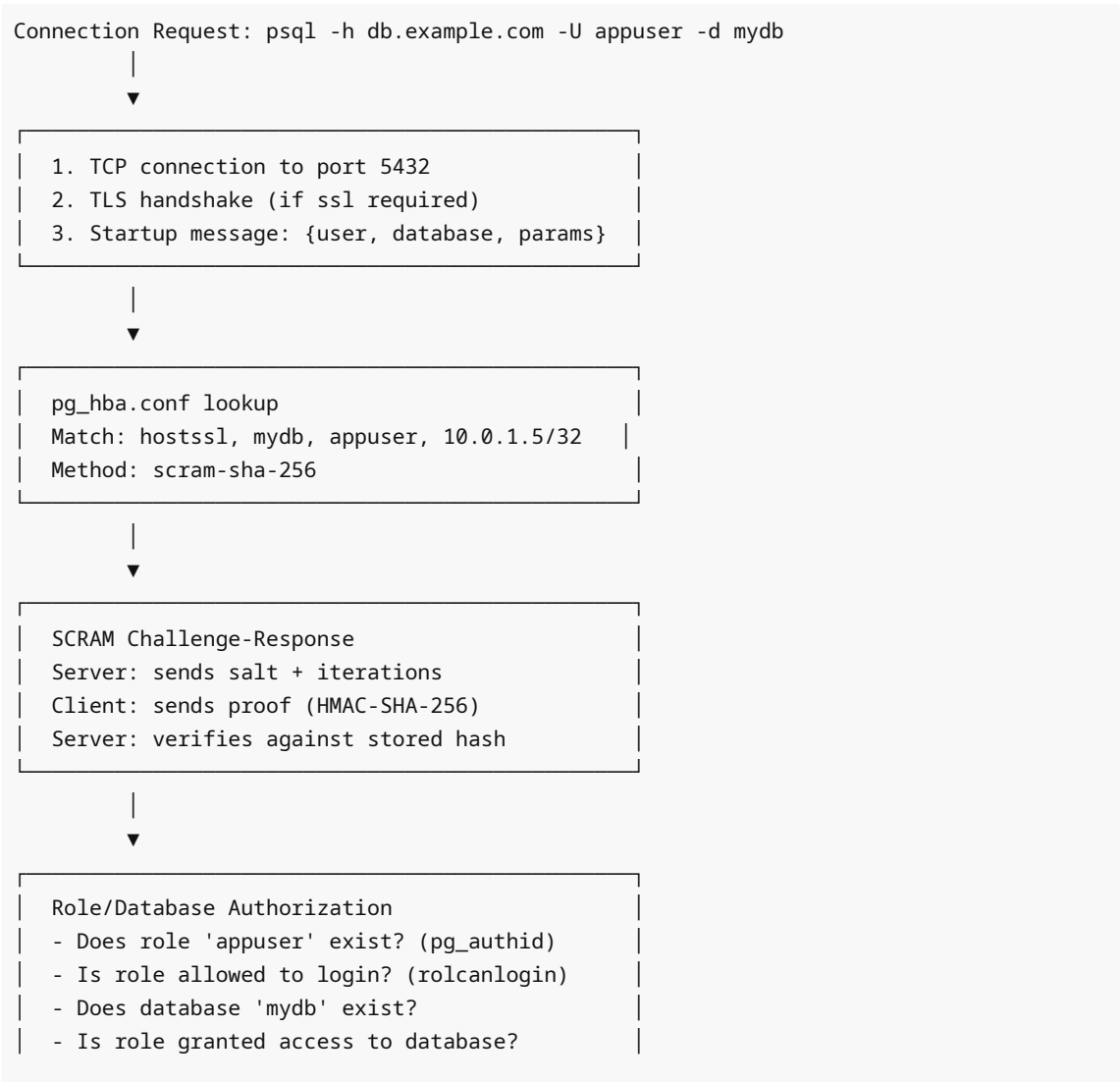
PAM Authentication

PAM (Pluggable Authentication Modules) allows PostgreSQL to use system authentication.

```
# pg_hba.conf
host all all 127.0.0.1/32 pam pamservice=postgresql
```

```
# /etc/pam.d/postgresql
auth include system-auth
account include system-auth
```

Authentication Flow Diagram



```
| - Is role's connection limit reached? |
```



Connection established / Rejected with error

Common Mistakes

1. **Trust on external interfaces** — anyone can connect without password
 2. **Using md5 instead of scram-sha-256** — weaker security
 3. **pg_hba.conf order matters!** — more specific entries must come before `all` catch-alls
 4. **No `reject` at end** — allows unexpected connections if no match
 5. **Not requiring SSL for remote connections** — password sent in cleartext (with md5)
 6. **Forgetting to reload after changes** — old rules stay active
 7. **Case sensitivity in usernames** — PostgreSQL usernames are case-folded to lowercase (unless quoted)
 8. **LDAP role not created** — LDAP verifies password but role must exist in PostgreSQL
 9. **Certificate CN mismatch** — cert auth fails if CN doesn't match username
 10. **Not testing changes before applying** — use `pg_hba_file_rules` to preview
-

Best Practices

1. Use **scram-sha-256** for all password-based authentication
 2. Require **SSL (`hostssl`)** for all remote connections
 3. Use **`reject`** as the last rule to explicitly deny unmatched connections
 4. Use **least privilege** — specific user/database/IP rather than `all`
 5. Never use **`trust`** on any non-loopback address
 6. Consider **certificate auth** for service accounts and automated processes
 7. Enable **`log_connections = on`** to audit all connection attempts
 8. Use **LDAP or RADIUS** for human users to leverage enterprise identity management
 9. Regularly **audit `pg_hba.conf`** — remove stale entries
 10. Test changes with **`pg_hba_file_rules`** before reloading
-

Performance Considerations

Authentication Method Performance (slowest to fastest):

- | | |
|-------------------------------|--|
| 1. <code>cert</code> (+ LDAP) | : Slowest (external call + TLS overhead) |
| 2. <code>scram-sha-256</code> | : Moderate (multiple hash rounds) |
| 3. <code>md5</code> | : Faster (simpler hash) |
| 4. <code>peer</code> | : Fast (OS call only) |
| 5. <code>trust</code> | : Fastest (no auth work) |

For high-connection-rate applications:

- Use connection pooling (PgBouncer) — most auth happens at PgBouncer level
 - SCRAM overhead per connection: <1ms — negligible with pooling
 - Without pooling: SCRAM at 10,000 connections/sec ≈ measurable overhead
-

Interview Questions

Q1: How does `pg_hba.conf` rule matching work?

A: PostgreSQL reads `pg_hba.conf` from top to bottom and uses the FIRST matching record. A record matches if the connection type (local/host/hostssl), database, username, and client IP all match. Once a match is found, that method is used — if authentication fails, PostgreSQL does NOT try the next rule. There is no fallback. This is why order is critical and a catch-all `reject` at the end is recommended.

Q2: What is the difference between `md5` and `scram-sha-256`?

A: MD5 stores and transmits passwords as MD5 hashes — vulnerable to offline dictionary attacks and rainbow tables. SCRAM-SHA-256 uses salted HMAC-SHA-256 with multiple iterations, preventing offline attacks and providing mutual authentication (client verifies server identity too). SCRAM never transmits the password or a static hash — only a cryptographic proof valid for one session.

Q3: What is required for certificate authentication to work?

A: (1) PostgreSQL server must have SSL enabled with `ssl=on`. (2) Server must have `ssl_ca_file` pointing to the CA that signed client certs. (3) `pg_hba.conf` must have `hostssl ... cert` for the connection. (4) The client certificate's CN must match the PostgreSQL username (or use `pg_ident.conf` for mapping). (5) The client certificate must be signed by the trusted CA.

Q4: Can PostgreSQL authenticate users that exist only in LDAP?

A: No — the user (role) must exist in PostgreSQL (`pg_authid`) even with LDAP authentication. LDAP only handles password verification. The user must be created with `CREATE ROLE username WITH LOGIN;` (no password needed since LDAP provides it). LDAP does not automatically create PostgreSQL roles.

Q5: What is `peer` authentication and when is it used?

A: Peer authentication is for local (Unix socket) connections only. It verifies that the client's OS username matches the PostgreSQL username. The PostgreSQL server queries the OS (via `getpeereid()`) for the connecting process's user ID. Used for `postgres` OS user connecting as `postgres` role — no password needed when the OS identity proves the claim.

Q6: How do you prevent password sniffing when using password authentication?

A: Use `hostssl` (require TLS) instead of `host` in `pg_hba.conf`. With `hostssl`, the TLS layer encrypts the entire connection including the SCRAM authentication exchange. With plain `host` and SCRAM, the protocol is still resistant to offline attack (SCRAM doesn't expose the password), but data after auth is unencrypted.

Q7: What does `+rolename` mean in the `USER` field of `pg_hba.conf`?

A: The `+` prefix means "this role and all roles that are members of this role." For example, `+analysts` allows connections by any user who is a member of the `analysts` role (directly or transitively). This is useful for group-based access control — add users to the group rather than editing `pg_hba.conf`.

Q8: How do you audit failed authentication attempts?

A: Enable `log_connections = on` and `log_disconnections = on` in `postgresql.conf`. Failed authentications are logged at `LOG` level with the message "connection received" followed by "authentication failed for user". These appear in the PostgreSQL log file. Also consider `log_line_prefix = '%t [%p]: [%l-1] user=%u,db=%d,app=%a,client=%h '` for context.

Exercises and Solutions

Exercise 1: Write pg_hba.conf for This Scenario

Requirements:

- Local postgres user: passwordless (OS auth)
- Monitoring tool on 192.168.10.5: password auth (monitoring_user)
- Application servers on 10.0.1.0/24: SSL + SCRAM
- Analytics team on 10.0.2.0/24: SSL + certificate auth
- Replication from 10.0.0.0/8: SSL + SCRAM (replicator user)
- Everything else: reject

```
# /etc/postgresql/16/main/pg_hba.conf

# Local
local all postgres peer
local all all scram-sha-256

# Monitoring
host all monitoring_user 192.168.10.5/32 scram-sha-256

# Application servers (SSL required)
hostssl all appuser 10.0.1.0/24 scram-sha-256

# Analytics (certificate)
hostssl analytics_db all 10.0.2.0/24 cert

# Replication
hostssl replication replicator 10.0.0.0/8 scram-sha-256

# Reject all others
host all all 0.0.0.0/0 reject
```

Exercise 2: Audit Users Not Using SCRAM

```
SELECT
    rolname,
    CASE
        WHEN passwd LIKE 'SCRAM-SHA-256%' THEN 'scram-sha-256'
        WHEN passwd LIKE 'md5%' THEN 'md5 (UPGRADE NEEDED)'
        WHEN passwd IS NULL THEN 'no password (peer/cert/trust)'
        ELSE 'unknown'
    END AS auth_type,
    rolvaliduntil AS password_expires
FROM pg_authid
WHERE rolcanlogin = true
ORDER BY auth_type, rolname;
```

Cross-References

- [02_roles_and_privileges.md](#) — Role management
- [04_ssl_tls.md](#) — SSL/TLS configuration details
- [07_secrets_management.md](#) — Password storage and rotation
- [06_auditing.md](#) — Auditing authentication events